

C++ Project Setup Guide - VS Code + MinGW

Table of Contents

- [Prerequisites](#prerequisites)
 - [Project Structure](#project-structure)
 - [Create Project Files](#create-project-files)
 - [VS Code Configuration](#vs-code-configuration)
 - [Build & Run](#build--run)
 - [Debug](#debug)
 - [Testing with Google Test & Mocks](#testing-with-google-test--mocks)
 - [Troubleshooting](#troubleshooting)
-

Prerequisites

Install MinGW (g++ compiler)

- Download MinGW from <https://www.mingw-w64.org/>
- Install to C:\MinGW
- Add to system PATH:
- Open System Properties Environment Variables
- Edit Path Add C:\MinGW\bin

Verify Installation

Open Command Prompt and run:

```
g++ --version
make --version
```

Install VS Code Extensions

```
ms-vscode.cpptools          # C/C++ IntelliSense
```

Press Ctrl+Shift+X Search and install above extensions.

Project Structure

```
ProjectName/  
  .vscode/  
    tasks.json  
    launch.json  
    c_cpp_properties.json  
  inc/          # Header files (.h, .hpp)  
  src/          # Source files (.cpp)  
  obj/          # Object files (.o)  
  bin/          # Executables (.exe)  
  lib/          # Libraries  
  Makefile
```

Create Folders

```
# Run in PowerShell  
New-Item -ItemType Directory -Path "inc","src","obj","bin","lib",".vscode" -Force
```

Create Project Files

1. Header File: `inc/calculator.h`

```
#ifndef CALCULATOR_H  
#define CALCULATOR_H  
  
class Calculator {  
public:  
    Calculator();  
    ~Calculator();  
    double add(double a, double b);  
    double subtract(double a, double b);  
    double multiply(double a, double b);  
    double divide(double a, double b);  
};  
  
#endif
```

2. Implementation: `src/calculator.cpp`

CppCalculator - Beginner Guide

```
#include <calculator.h>          // Angular brackets work via -I inc

Calculator::Calculator() {}
Calculator::~Calculator() {}

double Calculator::add(double a, double b) {
    return a + b;
}

double Calculator::subtract(double a, double b) {
    return a - b;
}

double Calculator::multiply(double a, double b) {
    return a * b;
}

double Calculator::divide(double a, double b) {
    if (b == 0) return 0;
    return a / b;
}
```

3. Main File: `src/main.cpp`

```
#include <iostream>
#include <calculator.h>          // Angular brackets work via -I inc

int main() {
    Calculator calc;
    std::cout << "5 + 3 = " << calc.add(5, 3) << std::endl;
    std::cout << "10 - 4 = " << calc.subtract(10, 4) << std::endl;
    return 0;
}
```

4. Makefile

```
# Compiler and flags
CXX = g++
CXXFLAGS = -Wall -Wextra -g -I inc
LDFLAGS =

# Directories
SRC_DIR = src
INC_DIR = inc
OBJ_DIR = obj
BIN_DIR = bin

# Target
TARGET = $(BIN_DIR)/calculator.exe

# Source and object files
SOURCES = $(wildcard $(SRC_DIR)/*.cpp)
OBJECTS = $(patsubst $(SRC_DIR)/%.cpp,$(OBJ_DIR)/%.o,$(SOURCES))

# Default target
all: $(TARGET)

# Link object files
$(TARGET): $(OBJECTS) | $(BIN_DIR)
    $(CXX) $(LDFLAGS) $^ -o $@

# Compile source files
$(OBJ_DIR)/%.o: $(SRC_DIR)/%.cpp | $(OBJ_DIR)
    $(CXX) $(CXXFLAGS) -c $< -o $@

# Create directories
$(BIN_DIR):
    mkdir -p $(BIN_DIR)

$(OBJ_DIR):
    mkdir -p $(OBJ_DIR)

# Clean
clean:
    rm -rf $(OBJ_DIR)/*.o $(BIN_DIR)/*.exe

# Run
run: $(TARGET)
    ./$$(TARGET)

.PHONY: all clean run
```

All configuration files go in the .vscode/ folder inside your project.

1. tasks.json - Build Tasks Configuration

This file tells VS Code how to compile/build your project.

```
{
  "version": "2.0.0",           // tasks.json format version (use 2.0.0)
  "tasks": [                   // Array of build tasks
    {
      // TASK 1: BUILD
      "label": "Build with Make", // Name shown in VS Code task list
      "type": "shell",           // Run as shell command
      "command": "C:/MinGW/bin/make.exe", // Path to make.exe
      "args": ["all"],           // Arguments: make all
      "group": {
        "kind": "build",         // Group type: "build" | "test" | "none"
        "isDefault": true       // This runs when you press Ctrl+Shift+B
      },
      "problemMatcher": ["$gcc"], // Parse g++ errors/warnings
      "detail": "Build project using Makefile" // Description
    },

    // TASK 2: CLEAN
    {
      "label": "Clean Build",     // Task name
      "type": "shell",           // Run as shell command
      "command": "C:/MinGW/bin/make.exe", // Path to make.exe
      "args": ["clean"],         // Arguments: make clean
      "group": "build",          // Part of build group
      "problemMatcher": []       // No error parsing needed
    },

    // TASK 3: RUN
    {
      "label": "Run Program",     // Task name
      "type": "shell",           // Run as shell command
      "command": "C:/MinGW/bin/make.exe", // Path to make.exe
      "args": ["run"],           // Arguments: make run
      "group": "test",           // Part of test group
      "problemMatcher": []       // No error parsing needed
    }
  ]
}
```

Field	Values	Description
-------	--------	-------------

CppCalculator - Beginner Guide

version	"2.0.0"	Format version, always use 2.0
label	Any string	Display name in VS Code task I
type	"shell"	Command type (shell or process
command	Path/string	The executable to run
args	Array of strings	Command-line arguments
group.kind	"build", "test", "none"	Task category
group.isDefault	true/false	Run on Ctrl+Shift+B
problemMatcher	["\$gcc"]	Parse compiler errors (clickab
detail	Any string	Optional description

```
{
  "label": "Run without rebuild",
  "type": "shell",
  "command": "./bin/calculator.exe",
  "group": "test",
  "problemMatcher": []
}
```

2. launch.json - Debug Configuration

This file tells VS Code how to debug your program.

```

{
  "version": "0.2.0",           // launch.json format version (use 0.2.0)
  "configurations": [          // Array of debug configurations
    {
      // DEBUGGER SETTINGS
      "name": "Launch Calculator", // Name shown in debug dropdown

      "type": "cppvsdbg",         // Debugger type:
                                   //   "cppvsdbg" = Visual Studio debugger (Windows)
                                   //   "cppdbg"   = GDB/MI debugger (MinGW)
                                   //   "lldb"    = LLDB debugger (macOS/Linux)

      "request": "launch",        // "launch" = start new process
                                   // "attach" = attach to running process

      // PROGRAM SETTINGS
      "program": "${workspaceFolder}/bin/calculator.exe", // Path to .exe
                                                         // ${workspaceFolder} = project root folder

      "args": [],                 // Command-line args passed to program
                                   // Example: ["--verbose", "--file=test.txt"]

      "cwd": "${workspaceFolder}", // Working directory when running

      "environment": [],          // Environment variables
                                   // Example: [{"name": "PATH", "value": "/usr/bin"}]

      // CONSOLE SETTINGS
      "console": "externalTerminal", // Where to show output:
                                       //   "externalTerminal" = new cmd window
                                       //   "integratedTerminal" = VS Code terminal
                                       //   "internalConsole" = Debug Console (no input)

      // BEHAVIOR
      "stopAtEntry": false,        // true = stop at main() automatically
                                   // false = run normally until breakpoint

      // BUILD BEFORE DEBUG
      "preLaunchTask": "Build with Make" // Run this task before debugging
                                         // Must match a task label in tasks.json
    }
  ]
}

```

Field	Values	Description
version	"0.2.0"	Format version, always use 0.2
name	Any string	Configuration name in debug dr

CppCalculator - Beginner Guide

type	"cppvsdbg", "cppdbg", "lldb"	Debugger type
request	"launch", "attach"	Start new or attach to running
program	Path to .exe	Executable to debug
args	Array of strings	Program arguments
cwd	Path	Working directory
environment	Array of objects	Environment variables
console	"externalTerminal", "integrate"	Output location
stopAtEntry	true/false	Stop at main()
preLaunchTask	Task label string	Task to run before debug

```
{
  "configurations": [
    {
      "name": "Debug (External Console)",
      "type": "cppvsdbg",
      "request": "launch",
      "program": "${workspaceFolder}/bin/calculator.exe",
      "console": "externalTerminal",
      "preLaunchTask": "Build with Make"
    },
    {
      "name": "Debug (Integrated Terminal)",
      "type": "cppvsdbg",
      "request": "launch",
      "program": "${workspaceFolder}/bin/calculator.exe",
      "console": "integratedTerminal",
      "preLaunchTask": "Build with Make"
    }
  ]
}
```

3. c_cpp_properties.json - IntelliSense Configuration

This file tells VS Code's code completion and error checking where to find headers.


```

{
  "configurations": [           // Array of configurations
    {
      // CONFIGURATION NAME
      "name": "MinGW",          // Name shown in bottom-right of VS Code
                                   // Click to switch configurations

      // INCLUDE PATHS
      "includePath": [
        "${workspaceFolder}/**", // Search all folders recursively
        "${workspaceFolder}/inc" // Your custom headers folder
      ],
      // This allows: #include <calculator.h> to find inc/calculator.h

      // PREDEFINED MACROS
      "defines": [
        "_DEBUG",               // Defined in debug builds
        "UNICODE",              // Unicode character support
        "_UNICODE"              // Unicode character support
      ],
      // Equivalent to: g++ -D_DEBUG -DUNICODE -D_UNICODE

      // COMPILER PATH
      "compilerPath": "C:/MinGW/bin/g++.exe", // Path to g++
      // Used by IntelliSense to determine:
      //   - System include paths
      //   - Default compiler flags
      //   - C/C++ standard version

      // LANGUAGE STANDARD
      "cStandard": "c11",        // C standard: c89, c99, c11, c17
      "cppStandard": "c++17",    // C++ standard: c++98, c++11, c++14, c++17, c++20

      // INTELLISENSE MODE
      "intelliSenseMode": "windows-gcc-x64" // Mode for Windows + GCC:
                                              // "windows-gcc-x64" = 64-bit MinGW
                                              // "windows-gcc-x86" = 32-bit MinGW
                                              // "windows-msvc-x64" = Visual Studio
    }
  ],
  "version": 4                  // Config file version (use 4)
}

```

Field	Values	Description
version	4	Config format version
name	Any string	Configuration name

CppCalculator - Beginner Guide

includePath	Array of paths	Where to find header files
defines	Array of strings	Preprocessor macros
compilerPath	Path to g++/clang++	Used for system headers
cStandard	"c89", "c99", "c11", "c17"	C language standard
cppStandard	"c++98" to "c++20"	C++ language standard
intelliSenseMode	See below	Platform + compiler mode

Mode	Platform	Compiler
windows-gcc-x64	Windows	MinGW 64-bit
windows-gcc-x86	Windows	MinGW 32-bit
windows-msvc-x64	Windows	Visual Studio
windows-clang-x64	Windows	Clang
linux-gcc-x64	Linux	GCC
macos-clang-x64	macOS	Clang

4. How It All Connects

VS CODE

c_cpp_properties.json

```
includePath: ["${workspaceFolder}/inc"]  
compilerPath: "C:/MinGW/bin/g++.exe"
```

IntelliSense finds: #include <calculator.h>

inc/calculator.h Found via includePath

tasks.json

```
Ctrl+Shift+B "Build with Make" task  
C:/MinGW/bin/make.exe all  
Compiles .cpp files to .o files  
Links to bin/calculator.exe
```

launch.json

```
F5 preLaunchTask: "Build with Make"  
  (runs task from tasks.json first)  
  
  Launches: bin/calculator.exe  
  Debugger: cppvsdbg (Windows native)  
  Console: externalTerminal (new cmd window)
```

5. Customization Examples

```
// In tasks.json:  
"command": "C:/my-libs/mingw/bin/make.exe"  
  
// In c_cpp_properties.json:  
"compilerPath": "C:/my-libs/mingw/bin/g++.exe"
```

```
"includePath": [  
    "${workspaceFolder}/**",  
    "${workspaceFolder}/inc",  
    "${workspaceFolder}/third_party/include"  
]
```

```
// In launch.json:  
"console": "integratedTerminal" // Use VS Code terminal (can type input)
```

```
// In launch.json:  
"args": ["--input", "data.txt", "--verbose"]
```

```
// In launch.json:  
"stopAtEntry": true // Debugger pauses at first line of main()
```

Build & Run

Method 1: VS Code

Action	Shortcut
Build	Ctrl+Shift+B
Run	F5

Method 2: Terminal

```
# Build
C:/MinGW/bin/make.exe all

# Run
./bin/calculator.exe

# Clean
C:/MinGW/bin/make.exe clean
```

Debug

- Open a .cpp file
- Click left of line number to set breakpoint (red dot)
- Press F5 to start debugging

Debug Controls

Button	Action
Continue	Run until next breakpoint
Step Over	Execute next line
Step Into	Enter function
Step Out	Exit current function
Restart	Restart debugging
Stop	Stop debugging

Testing with Google Test & Mocks

This project uses Google Test (gtest) for unit testing and Google Mock (gmock) for creating mock objects.

Project Structure (Updated)

```
CppCalculator/  
  .vscode/  
    tasks.json  
    launch.json  
    c_cpp_properties.json  
  inc/  
    ICalculator.h          # Interface (abstract class)  
    calculator.h          # Concrete implementation  
    MockCalculator.h       # Mock class for testing  
  src/  
    calculator.cpp        # Implementation  
    main.cpp             # Application entry  
  tests/  
    CalculatorTest.cpp    # Unit tests  
  third_party/  
    googletest/           # Google Test framework  
      build/              # Built libraries  
      googletest/         # gtest source  
      gmockmock/          # gmock source  
  obj/  
  bin/  
  lib/  
  Makefile
```

Architecture Overview

ICalculator (Interface)

```
+ add(a, b) = 0  
+ subtract(a, b) = 0  
+ multiply(a, b) = 0  
+ divide(a, b) = 0  
+ power(base, exp) = 0  
+ squareRoot(value) = 0
```

Calculator
(Real Impl)

MockCalculator
(Test Double)

Interface Pattern (ICalculator.h)

CppCalculator - Beginner Guide

The interface enables dependency injection and mocking:

```
class ICalculator {
public:
    virtual ~ICalculator() = default;
    virtual double add(double a, double b) = 0;
    virtual double subtract(double a, double b) = 0;
    // ... other pure virtual functions
};
```

Benefits:

- Swap implementations at runtime
- Create test doubles (mocks) for isolated testing
- Follow SOLID principles (Dependency Inversion)

Mock Class (MockCalculator.h)

```
class MockCalculator : public ICalculator {
public:
    MOCK_METHOD(double, add, (double a, double b), (override));
    MOCK_METHOD(double, subtract, (double a, double b), (override));
    // ... other mock methods
};
```

Writing Unit Tests

```
#include <gtest/gtest.h>
#include <calculator.h>

class CalculatorTest : public ::testing::Test {
protected:
    Calculator calc; // Setup code here if needed
};

TEST_F(CalculatorTest, Add_PositiveNumbers) {
    EXPECT_DOUBLE_EQ(calc.add(2.0, 3.0), 5.0);
}

TEST_F(CalculatorTest, Divide_ByZero_ThrowsException) {
    EXPECT_THROW(calc.divide(10.0, 0.0), std::runtime_error);
}
```

Use TEST_F with a fixture class for shared setup:

```
class CalculatorTest : public ::testing::Test {
protected:
    void SetUp() override {
        // Runs before each test
        calc = Calculator();
    }

    void TearDown() override {
        // Runs after each test
    }

    Calculator calc;
};
```

Assertion	Description
EXPECT_EQ(a, b)	a == b
EXPECT_DOUBLE_EQ(a, b)	a == b (floating point)
EXPECT_THROW(stmt, Exception)	stmt throws Exception
EXPECT_NO_THROW(stmt)	stmt doesn't throw
EXPECT_TRUE(cond)	cond is true
EXPECT_THAT(val, matcher)	val matches matcher

Use ASSERT_* versions to abort test on failure (instead of continuing).


```
#include <gmock/gmock.h>
#include <MockCalculator.h>

class MockCalculatorTest : public ::testing::Test {
protected:
    MockCalculator mockCalc;
};

TEST_F(MockCalculatorTest, Add_Expectation) {
    EXPECT_CALL(mockCalc, add(2.0, 3.0))
        .WillOnce(Return(5.0));

    double result = mockCalc.add(2.0, 3.0);
    EXPECT_DOUBLE_EQ(result, 5.0);
}

TEST_F(MockCalculatorTest, VerifyCallCount) {
    EXPECT_CALL(mockCalc, multiply(::testing::__, ::testing::_))
        .Times(3)
        .WillRepeatedly(Return(100.0));

    mockCalc.multiply(1.0, 2.0);
    mockCalc.multiply(3.0, 4.0);
    mockCalc.multiply(5.0, 6.0);
}
```

Matcher	Description
<code>::testing::_</code>	Any argument
<code>Eq(val)</code>	Equal to val
<code>Lt(val)</code>	Less than val
<code>Gt(val)</code>	Greater than val
<code>Between(a, b)</code>	Between a and b

```
// Production code uses interface
class CalculatorService {
public:
    explicit CalculatorService(ICalculator& calculator)
        : m_calculator(calculator) {}

    double computeAverage(double a, double b) {
        double sum = m_calculator.add(a, b);
        return m_calculator.divide(sum, 2.0);
    }

private:
    ICalculator& m_calculator;
};

// Test with mock
TEST(CalculatorServiceTest, ComputeAverage) {
    MockCalculator mockCalc;
    CalculatorService service(mockCalc);

    EXPECT_CALL(mockCalc, add(4.0, 6.0)).WillOnce(Return(10.0));
    EXPECT_CALL(mockCalc, divide(10.0, 2.0)).WillOnce(Return(5.0));

    EXPECT_DOUBLE_EQ(service.computeAverage(4.0, 6.0), 5.0);
}
```

Build Commands

Command	Description
mingw32-make test	Build and run all tests
mingw32-make test-verbose	Run tests with detailed output
mingw32-make test-add	Run only Add tests
mingw32-make test-mock	Run only Mock tests
mingw32-make all-with-tests	Build tests then app

Test Results Example

```
[=====] Running 36 tests from 3 test suites.  
[-----] 28 tests from CalculatorTest  
[ RUN      ] CalculatorTest.Add_PositiveNumbers  
[          OK ] CalculatorTest.Add_PositiveNumbers (0 ms)  
...  
[-----] 6 tests from MockCalculatorTest  
[ RUN      ] MockCalculatorTest.Add_Expectation  
[          OK ] MockCalculatorTest.Add_Expectation (0 ms)  
...  
[-----] 2 tests from CalculatorServiceTest  
[ RUN      ] CalculatorServiceTest.ComputeAverage  
[          OK ] CalculatorServiceTest.ComputeAverage (0 ms)  
...  
[ PASSED   ] 36 tests.
```

Running Tests in VS Code

Add to .vscode/tasks.json:

```
{  
  "label": "Run Tests",  
  "type": "shell",  
  "command": "C:/MinGW/bin/mingw32-make.exe",  
  "args": ["test"],  
  "group": {  
    "kind": "test",  
    "isDefault": true  
  },  
  "problemMatcher": ["$gcc"]  
}
```

Then press Ctrl+Shift+T to run tests.

Test Coverage Best Practices

- Test all public methods - Every function in ICalculator has tests
- Test edge cases - Zero, negative, large numbers, infinity
- Test exceptions - Verify error handling works correctly
- Test mock expectations - Ensure mocks are called correctly
- Use descriptive test names - CalculatorTest.Divide_ByZero_ThrowsException

Troubleshooting

Issue: "g++ not found"

Fix: Add MinGW to PATH:

```
# Temporary (current session)
$env:Path += ";C:\MinGW\bin"

# Permanent
[System.Environment]::SetEnvironmentVariable("Path", $env:Path + ";C:\MinGW\bin", "User")
```

Issue: "Unable to start debug"

Fix: Use cppvsdbg instead of cppdbg in launch.json:

```
"type": "cppvsdbg"
```

Issue: Custom headers not found

Fix: Ensure c_cpp_properties.json has:

```
"includePath": ["${workspaceFolder}/inc"]
```

And Makefile has -I inc flag.

Issue: Makefile tabs error

Fix: Makefile requires tabs, not spaces for indentation.

- Open Makefile in VS Code
- Enable: View Render Whitespace
- Ensure recipe lines start with Tab character

Issue: Red squiggly lines under headers

Fix: Press Ctrl+Shift+P C/C++: Reset IntelliSense Database

Quick Reference

Task	Command
------	---------

CppCalculator - Beginner Guide

Build	Ctrl+Shift+B
Debug	F5
Clean	Terminal: make clean
Add new .cpp	Create in src/, rebuild
Add new .h	Create in inc/, include with <

Adding New Files

- Create header in inc/newfile.h
- Create source in src/newfile.cpp
- Include header using angular brackets:

```
`cpp
```

```
#include <newfile.h>
```

```
`
```

- Press Ctrl+Shift+B to rebuild

Makefile automatically detects new .cpp files in src/.

Guide created for CppCalculator project